

Name: Student ID:

ECS401: Unit 2 Lab Progress Test

This sheet is to help you and us judge your progress. You must complete it at the start of your lab session before any other activity (including getting work marked). Compare notes with others nearby and make sure everyone understands the right answers (available on the QM+ site). Talk to a demonstrator about anything that is at all unclear. Do NOT spend more than 15 minutes on answering the questions: if you don't know, leave it and get someone to explain.

In each question below, assume the code fragment is inside the body of a method.

1. What is the value in the variable `answer` after the following fragment of code has been executed?

```
int answer = 55;
```

Answer:

For questions 2 and 3 first consider the following fragment of code and complete the dry run table:

```
int a;  
int b;  
a = 2; // L1  
b = 3; // L2  
b = a; // L3
```

Line	a	b

2. What is the value in the variable `a` after the above fragment of code has been executed?

Answer:

3. What is the value in the variable `b` after the above fragment of code has been executed?

Answer:

4. What is the value in the variable `c` after the above fragment of code has been executed?

```
int c;
```

Answer:

5. What is the value in the variable `d` after the above fragment of code has been executed?

```
int d = 2 + 3 * 4;
```

Answer:

6. What is printed to the screen after the above has been executed?

```
String e = "a";  
System.out.println("b" + e + "d");
```

Answer:

For questions 7 and 8, first consider the above fragment of code and complete the dry run table:

```
int f = 1; // L1  
int g = 2; // L2  
int h = 3; // L3  
h = f      // L4  
f = g;     // L5  
g = h;     // L6
```

Line	f	g	h

7. What is the value in variable `f` after the above has been executed?

Answer:

8. What is the value in variable `g` after the above has been executed?

Answer:

9. Write a fragment of code that declares 4 variables, initialising them to 10, 20, 30 and 40 and then stores their average in a fifth variable.

Answer:

Terminology

10. A **procedure call** is where a previously defined piece of code (a method) is executed by giving its name. Procedures *do not* return values. Highlight/underline all the places where procedure calls appear in the following code.

```
public static void main (String[] param)
{
    helloMessage();
    return;
} // END main

public static void helloMessage ()
{
    System.out.println("Hello World!");
    return;
} // END helloMessage
```

11. A **function call** is where a previously defined piece of code (a method) is executed by giving its name. Functions *do* return values. Highlight/underline all the places where function calls appear in the following code.

```
public static void main (String[] param)
{
    String message = helloMessage();
    System.out.println(message);
    return;
} // END main

public static String helloMessage ()
{
    String m = inputString("Enter a message");
    return m;
} // END helloMessage
```